Deep Learning - Activation Functions - Nonlinear Learning

Refai Arafat Rossi

October 2026

1 Introduction

[Chapter 6: Nonlinear Learning]Nonlinear Learning From Activation Functions to Backpropagation Master's Level Lecture Deck Department of Computer Science / Data Science October 10, 2026

---

height

1. Problem: Electricity Demand vs. Temperature Imagine an electricity grid operator planning daily power generation.

- **Cold days:** Demand rises due to electric space heating.
- **Hot days:** Demand rises due to air conditioning.
- **Comfortable days:** Demand is lowest around moderate temperatures.

Mathematical Formulation

- Input x: Temperature deviation from comfortable baseline.
- Target y: Additional electricity demand.
- Illustrative Ground Truth Model:

$$y = x^2$$

Sample Data Points:

Temp Deviation (x)	-2	-1	0	1	2
Demand ($y = x^2$)	4	1	0	1	4

2. Why Do We Need a Neural Network? 0.5 **Model A: Linear Regression**

$$\hat{y} = wx + b$$

Limitation Can only fit a straight line. Fails completely on U-shaped non-linear curves like $y = x^2$.

Model B: Single-Layer Neural Network

$$h = f(wx + b)$$

Non-linear activation $f(\cdot)$ combines piecewise signals to approximate curves.

0.5 **Model C: Deep Neural Network**

$$h^{(l)} = f\left(W^{(l)}h^{(l-1)} + b^{(l)}\right)$$

Multiple stacked layers construct hierarchical, highly complex functional representations.

Takeaway A linear model cannot capture parabolic demand. Hidden layers with non-linear activations provide universal approximation capabilities.

3. Activation Functions: ReLU and Sigmoid An artificial neuron calculates $z = wx + b$, then applies activation $a = f(z)$.

0.5 Rectified Linear Unit (ReLU)

$$ReLU(z) = \max(0, z)$$

- $ReLU(-3) = 0$
- $ReLU(2.5) = 2.5$
- Standard choice for hidden layers (simple derivative, avoids gradient saturation).

0.5 Sigmoid Activation

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

- $\sigma(0) = 0.5$
- $\sigma(2) \approx 0.881, \quad \sigma(-2) \approx 0.119$
- Maps $R \to (0, 1)$. Standard choice for binary classification outputs.

4. Network Architecture for Electricity Demand 0.5 **1-Hidden-Layer Regression Network: Hidden 1:** $z_1 = w_1 x + b_1, \quad h_1 = ReLU(z_1)$
Hidden 2: $z_2 = w_2 x + b_2, \quad h_2 = ReLU(z_2)$
Output: $\hat{y} = v_1 h_1 + v_2 h_2 + c$

Vector Matrix Form

$$z^{(1)} = W^{(1)}x + b^{(1)}$$
$$h^{(1)} = ReLU\left(z^{(1)}\right)$$
$$\hat{y} = W^{(2)}h^{(1)} + b^{(2)}$$

0.5 [x=1.6cm, y=1.2cm, ¿=stealth] [circle, draw, fill=blue!10] (x) at (0, 0) x;
[circle, draw, fill=green!10] (h1) at (1.5, 0.8) h_1; [circle, draw, fill=green!10] (h2) at (1.5, -0.8) h_2;
[circle, draw, fill=red!10] (y) at (3, 0) $\hat{y}$;
[-¿] (x) – node[above left] w_1, b_1 (h1); [-¿] (x) – node[below left] w_2, b_2 (h2); [-¿] (h1) – node[above right] v_1 (y); [-¿] (h2) – node[below right] v_2, c (y);

5 & 6. Part B — Numerical Forward Pass Example **Training Instance:** $x = 1, \quad TrueTargety = 1.$

Initial Parameters:

$$w_1 = 1,\ b_1 = 0 \quad | \quad w_2 = 0.5,\ b_2 = 0.5 \quad | \quad v_1 = 0.2,\ v_2 = 0.2,\ c = 0$$

0.5 **Hidden Neuron 1:**

- $z_1 = (1)(1) + 0 = 1$
- $h_1 = ReLU(1) = \mathbf{1}$

Hidden Neuron 2:

- $z_2 = (0.5)(1) + 0.5 = 1$
- $h_2 = ReLU(1) = \mathbf{1}$

0.5 **Output Neuron ($\hat{y}$):**

$$\hat{y} = (0.2)(1) + (0.2)(1) + 0 = \mathbf{0.4}$$

Prediction Error

$$e = \hat{y} - y = 0.4 - 1.0 = \mathbf{-0.6}$$

The network underpredicted the true target (0.4 vs. 1.0).

7. Calculate the Loss To evaluate model error for differentiation, we use the **half-squared error** for a single sample:

$$L = \frac{1}{2}(\hat{y} - y)^2$$

Numerical Evaluation:

$$L = \frac{1}{2}(0.4 - 1.0)^2 = \frac{1}{2}(-0.6)^2 = \frac{1}{2}(0.36) = \mathbf{0.18}$$

General Mean Squared Error (MSE) across m samples

$$MSE = \frac{1}{m}\sum_{i=1}^{m}(\hat{y}_i - y_i)^2$$

The factor of $\frac{1}{2}$ cancels out neatly during derivative operations.

8. Backpropagation: Calculating Gradients via Chain Rule Backpropagation propagates error backward from output to input to calculate $\frac{\partial L}{\partial \theta}$.

Step 1: Gradient at Output ($\hat{y}$)

$$\frac{\partial L}{\partial \hat{y}} = \hat{y} - y = 0.4 - 1.0 = \mathbf{-0.6}$$

Step 2: Gradients for Output Layer Weights (v_1, v_2) and Bias (c)

$$\frac{\partial L}{\partial v_1} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial v_1} = (-0.6)(h_1) = (-0.6)(1) = \mathbf{-0.6}$$

$$\frac{\partial L}{\partial v_2} = (-0.6)(h_2) = \mathbf{-0.6}, \qquad \frac{\partial L}{\partial c} = \frac{\partial L}{\partial \hat{y}}(1) = \mathbf{-0.6}$$

Step 3: Gradients for Hidden Layer Parameters (w_1, b_1)

$$\frac{\partial L}{\partial h_1} = \frac{\partial L}{\partial \hat{y}} v_1 = (-0.6)(0.2) = -0.12 \frac{\partial L}{\partial z_1} = \frac{\partial L}{\partial h_1} \cdot ReLU'(1) = (-0.12)(1) = -0.12$$

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial z_1} x = (-0.12)(1) = \mathbf{-0.12}, \qquad \frac{\partial L}{\partial b_1} = \mathbf{-0.12}$$

9. Gradient Descent Parameter Update Update all parameters using learning rate $\eta = 0.1$ and rule $\theta_{new} = \theta_{old} - \eta \frac{\partial L}{\partial \theta}$:

0.5 **Hidden Layer Updates:**

- $w_{1,new} = 1.0 - (0.1)(-0.12) = \mathbf{1.012}$
- $w_{2,new} = 0.5 - (0.1)(-0.12) = \mathbf{0.512}$
- $b_{1,new} = 0.0 - (0.1)(-0.12) = \mathbf{0.012}$
- $b_{2,new} = 0.5 - (0.1)(-0.12) = \mathbf{0.512}$

0.5 **Output Layer Updates:**

- $v_{1,new} = 0.2 - (0.1)(-0.6) = \mathbf{0.26}$
- $v_{2,new} = 0.2 - (0.1)(-0.6) = \mathbf{0.26}$
- $c_{new} = 0.0 - (0.1)(-0.6) = \mathbf{0.06}$

10. Verifying Improvement (Forward Pass Step 2) Re-evaluate network predictions using updated parameter values at $x = 1$:

- $z_1 = (1.012)(1) + 0.012 = 1.024 h_1 = 1.024$
- $z_2 = (0.512)(1) + 0.512 = 1.024 h_2 = 1.024$
- New Prediction: $\hat{y}_{new} = 0.26(1.024) + 0.26(1.024) + 0.06 = \mathbf{0.5925}$
- New Loss: $L_{new} = \frac{1}{2}(0.5925 - 1.0)^2 = \mathbf{0.0830}$

Quantity	Before Update	After Update	Improvement
Prediction ($\hat{y}$)	0.4000	0.5925	Closer to target 1.0
Half-Squared Loss (L)	0.1800	0.0830	**Reduced by** 53.9%

11 & 12. Part C — Bias–Variance Trade-off 0.33 Underfitting

- Model capacity too low.
- E_{train} High, E_{test} High.
- High Systematic Bias.

0.33 Optimal Capacity

- Balanced complexity.
- E_{train} Low, E_{test} Low.
- Good Generalization.

0.33 Overfitting

- Capacity excessively high.
- $E_{train} \approx 0$, E_{test} High.
- High Variance.

Expected Generalization Error Decomposition

$$ExpectedError = Bias^2 + Variance + IrreducibleNoise$$

13. Part D — Classification: Sigmoid & Binary Cross-Entropy Shift task from regression to **Binary Classification** ($y \in \{0, 1\}$).

Step 1: Compute Logit to Probability Mapping via Sigmoid

$$p = \sigma(z) = \frac{1}{1+e^{-z}} \quad If z = 2.0: \quad p = \frac{1}{1+e^{-2}} \approx \mathbf{0.881} \quad (88.1\% probofClass1)$$

Step 2: Binary Cross-Entropy Loss (BCE)

$$L_{BCE} = -\left[y \log p + (1-y)\log(1-p)\right]$$

- If True Label $y = 1 L = -\log(p)$.
- If $p = 0.881 L = -\log(0.881) \approx \mathbf{0.127}$ (Low loss for correct confident prediction).

Task Type	Output Activation	Output Range	Loss Function
Continuous Regression	Linear / Identity ($f(z) = z$)	$(-\infty, \infty)$	Mean Squared Error (MSE)
Binary Classification	Sigmoid ($\sigma(z)$)	$(0, 1)$	Binary Cross-Entropy (BCE)
Multiclass Classification	Softmax (p_i)	$[0, 1]$, $\sum p_i = 1$	Categorical Cross-Entropy

14. Multiclass Classification: Softmax Categorical Cross-Entropy For K-class classification (e.g., MRI tumor diagnosis: Glioma, Meningioma, Pituitary):

1. Softmax Activation Function: Converts raw logits $z = [z_1, z_2, \ldots, z_K]^T$ into probabilities:

$$p_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

Numerical Example: Given network logits $z = [2.0, 1.0, 0.0]^T$:

- $e^{2.0} \approx 7.389, \quad e^{1.0} \approx 2.718, \quad e^{0.0} = 1.0 Sum = 11.107$
- $p = \left[\frac{7.389}{11.107}, \frac{2.718}{11.107}, \frac{1.0}{11.107}\right] = [\mathbf{0.665}, \mathbf{0.245}, \mathbf{0.090}]$

2. Categorical Cross-Entropy Loss: If True Class is Category 2 ($y = [0, 1, 0]^T$):

$$L_{CCE} = -\log(p_2) = -\log(0.245) \approx \mathbf{1.407}$$

15. Summary: Task vs. Output Layer Selection

Golden Rule Hidden layers use non-linear activations (e.g., ReLU) to learn representations; the output activation function is dictated strictly by the task type.

16. Core Equations Cheat Sheet 0.5

1. **Neuron Pre-activation:**

$$z = w^T x + b$$

2. **ReLU Activation:**

$$ReLU(z) = \max(0, z)$$

3. **Sigmoid Activation:**

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

0.5

4. **Softmax Function:**

$$p_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

5. **MSE Loss:**

$$MSE = \frac{1}{m} \sum_{i=1}^{m} (\hat{y}_i - y_i)^2$$

6. **Gradient Descent Update:**

$$\theta \leftarrow \theta - \eta \nabla_\theta L$$

17. Final Picture: The End-to-End Learning Loop

$$\text{Input } x \longrightarrow ForwardPass(\hat{y}) \longrightarrow LossL \longrightarrow Backprop(\nabla_\theta L) \longrightarrow GradientUpdate(\theta)$$

Final Summary Takeaway

- **Forward Pass:** Generates model predictions.
- **Loss Function:** Quantifies discrepancy against ground-truth targets.
- **Backpropagation:** Uses calculus chain rule to attribute error gradients to every parameter.
- **Gradient Descent:** Updates parameters to iteratively minimize loss and improve predictions.